재무 진단 기능
코드 해설 및 상세 리뷰

front/src/features/financial 및 연관 Financial 페이지

실제 소스 발췌 · 동작 해설 · 결함 분석 · 개선 예시 · 검증 계획

이 문서가 다루는 내용

함수 이름이나 문제 목록만 나열하지 않고, 현재 구현의 중요한 코드 구간을 줄 번호와 함께 읽습니다. 각 구간이 어떤 입력을 받고 어떤 값을 만드는지 설명한 뒤, 화면 간 데이터 흐름과 계산 결과가 어디에서 어긋나는지 연결합니다. 마지막에는 수정 방향을 보여주는 예시 코드와 검증 시나리오를 제공합니다.

구분	주요 내용
02-04	라우팅과 상태 전달, 소득·지출 입력, 재무 요약 함수
05-08	대출 관리, 예산 산식, P1 상환액 누락, P2 자산 검증
09-10	신용 입력 기본값, P2 미확인 정보 판정
11-13	차량 조건의 실제 사용 범위, 우선순위와 포인터 처리
14-16	확인·결과 화면, 통합 개선 예시, 검증 결과와 우선순위

리뷰 판단

확인된 결함은 P1 1건과 P2 2건입니다. 추가로 기본값의 의미, 입력 상태의 경계, 차량 조건의 사용 범위, 컴포넌트 책임과 같은 설계 관찰을 별도로 설명합니다. 설계 개선 제안과 미검증 위험을 확인된 결함으로 단정하지 않습니다.

근거와 한계

실제 코드는 문서 작성 시 로컬 소스에서 직접 추출했습니다. 금액 단위는 만 원입니다. 이전 검토에서 수행한 Node.js 계산 재현과 빌드 결과를 사용하며, 이번 문서 작성 중 브라우저 테스트나 제품 코드 수정은 수행하지 않았습니다. 예시 코드는 수정 방향을 설명하기 위한 제안입니다.

02 전체 구조와 상태 전달

실제 코드 | front/src/App.jsx:38-43

```
38          <Route path="/financial" element={<FinancialPage />} />
39          <Route path="/financial/assets" element={<FinancialAssetsPage />} />
40          <Route path="/financial/credit" element={<FinancialCreditPage />} />
41          <Route path="/financial/vehicle" element={<FinancialVehiclePage />} />
42          <Route path="/financial/review" element={<FinancialReviewPage />} />
43          <Route path="/financial/result" element={<FinancialResultPage />} />
```

여섯 개 경로가 입력 시작부터 결과까지를 각각 독립적인 페이지 컴포넌트에 연결합니다. 따라서 이 기능은 한 컴포넌트 안에서 단계만 바꾸는 구조가 아니라, 페이지 이동 시 데이터를 함께 넘겨야 하는 다단계 폼입니다.

실제 코드 | front/src/pages/FinancialPage.jsx:43-43

```
43    const [values, setValues] = useState(location.state?.values ?? {});
```

실제 코드 | front/src/pages/FinancialPage.jsx:52-55

```
52    function handleSubmit(event) {
53      event.preventDefault();
54      if (!event.currentTarget.reportValidity()) return;
55      navigate(location.state?.reviewReturn ? '/financial/review' : '/financial/assets', { state: { ...location.state, values, reviewReturn: false
    } });
```

상태가 이동하는 방식

43행은 전달받은 location.state.values로 로컬 입력 상태를 초기화합니다. 55행은 기존 state를 펼쳐서 자산·신용·차량 정보를 보존하고, 현재 페이지의 values를 최신 값으로 교체합니다. reviewReturn이 참이면 확인 화면으로 바로 복귀하고, 그렇지 않으면 다음 단계로 진행합니다.

state 필드	역할 / 주요 소비자
values	월 소득·지출·저축 / 재무 요약, 차량 예산
assets / loans	예금·선수금·비상금 / 자산 요약; 대출 상세는 현재 예산에서 누락
credit	연체·신규 대출·신용거래 정보 / 결과 안내
vehicle / priorities	이용 조건·선호 순서 / 확인 화면, 할부 예산, 이용 프로필
reviewReturn	확인 화면의 수정 후 돌아갈 경로 선택

구조에 대한 판단

기존 state를 보존하면서 해당 단계 값만 덮어쓰는 방식은 작은 폼에서 이해하기 쉽습니다. 다만 다른 페이지로 이동할 때의 명시적인 전달에 의존합니다. 새 탭이나 직접 URL 진입에서 state가 없을 수 있으므로 단계별 필수 데이터의 확인 정책을 통일하는 것이 좋습니다. 일반적인 새로고침만으로 반드시 state가 사라진다고 단정할 수는 없습니다.

03 소득·지출 입력과 기본 검증

실제 코드 | front/src/pages/FinancialPage.jsx:6-18

```
6  const fields = [
7    { name: 'income', label: '월 실수령액', required: true },
8    { name: 'housing', label: '월 주거비', required: true },
9    { name: 'fixed', label: '월 고정 지출', required: true },
10   { name: 'living', label: '월 생활비', required: true },
11   { name: 'loan', label: '현재 대출 월 상환액', required: false },
12   { name: 'savings', label: '월 목표 저축액', required: false },
13 ];
14
15 const extraFields = [
16   { name: 'extraIncome', label: '기타 정기 소득', required: false },
17   { name: 'transport', label: '교통비', required: false },
18   { name: 'phone', label: '통신비', required: false },
```

실제 코드 | front/src/pages/FinancialPage.jsx:66-78

```
66            <input
67              id={`financial-${name}`}
68              name={name}
69              type="number"
70              min="0"
71              step="0.01"
72              inputMode="decimal"
73              placeholder="0"
74              required={required}
75              value={values[name] ?? ''}
76              onChange={(event) => setValues((previous) => ({ ...previous, [name]: event.target.value }))}
77              aria-describedby="financial-unit"
78            />
```

코드의 의미

필드 정의에 name·label·required를 모아 동일한 입력 UI를 생성합니다. required=true인 소득·주거비·고정 지출·생활비는 비어 있으면 제출할 수 없습니다. 대출 상환·저축·기타 소득 등은 선택 입력입니다. value와 onChange가 연결되어 있어 입력값은 React 상태에서 관리됩니다.

number 입력의 min=0은 음수를, step=0.01은 소수 둘째 자리보다 세밀한 값을 브라우저 유효성 검사에서 제한합니다. event.target.value는 문자열이므로 계산 시 숫자로 바꾸는 책임은 별도로 필요합니다. 0은 유효한 값이고 빈 문자열은 미입력이므로 두 상태를 섞지 않는 것이 중요합니다.

잘된 점과 개선 지점

입력 정의와 렌더링의 반복을 줄였고 label과 입력 id도 연결되어 있습니다. 반면 대출 월 상환액은 다음 단계의 대출 상세 합계와 중복 표현됩니다. 입력 UI 자체보다, 두 값 중 어느 것이 최종 계산의 기준인지 정하지 않은 점이 핵심 문제입니다.

현재 제출 검증은 브라우저 기능을 사용합니다. 검증이 없다고 볼 수는 없지만, 프로젝트의 입력칸별 오류 표시 기준에 맞추려면 오류 상태와 설명을 보완할 수 있습니다. 함수가 API나 저장 데이터에서도 호출된다면 브라우저 검증과 별도로 숫자 계약을 확인해야 합니다.

04 재무 요약 함수의 계산 계약

실제 코드 | front/src/features/financial/calculateFinancialSummary.js:1-8

```
1  export function calculateFinancialSummary(values = {}) {
2    const amount = (name) => Number(values[name] || 0);
3    const income = amount('income') + amount('extraIncome');
4    const expenses = ['housing', 'fixed', 'living', 'loan', 'transport', 'phone']
5      .reduce((sum, name) => sum + amount(name), 0);
6    const savings = amount('savings');
7    return { income, expenses, savings, available: income - expenses - savings };
8  }
```

입력에서 결과까지

2행의 amount는 비어 있거나 없는 값을 0으로 처리한 뒤 Number로 변환합니다. 3행은 기본 소득과 기타 소득을 합산합니다. 4-5행은 주거비·고정 지출·생활비·대출 상환·교통비·통신비를 더합니다. 6행의 저축은 소비 지출과 분리하고, 7행에서 소득에서 지출과 목표 저축을 뺀 월 가용자금을 반환합니다.

예시 입력	합계
소득 300 + 기타 소득 20	income = 320
주거 50 + 고정 30 + 생활 50 + 대출 40 + 교통 10 + 통신 5	expenses = 185
목표 저축 50	savings = 50
320 - 185 - 50	available = 85

설계 평가

React나 DOM에 의존하지 않고 입력 객체만으로 결과를 만드는 순수 함수입니다. 실시간 계산, 확인 화면, 예산 계산이 같은 함수를 재사용하는 방향은 적절합니다. 계산 정책을 컴포넌트 내부에 복사하지 않아 검증과 수정 범위도 작습니다.

계약의 한계

이 함수가 읽는 대출은 values.loan 하나뿐입니다. loans 배열이 있어도 합산하지 않습니다. 또한 임의의 비숫자 문자열은 NaN으로, 무한대는 Infinity로 전파될 수 있습니다. 현재 정상 number 입력 경로와 구분해야 하지만, 계산 함수의 재사용 범위가 넓어질 때는 유한한 0 이상 숫자라는 전제를 코드로 명시하는 편이 좋습니다.

가용자금이 음수인 값을 그대로 반환하는 것은 적자 상태를 보여주기 위해 유용합니다. 이를 무조건 0으로 바꾸면 사용자 재무 상태가 숨겨질 수 있으므로, 표시용 가용자금과 대출 산정용 하한 처리는 구분해야 합니다.

05 자산·부채 상태와 상세 대출 관리

실제 코드 | front/src/pages/FinancialAssetsPage.jsx:24-43

```
24  export default function FinancialAssetsPage() {
25    const { state } = useLocation();
26    const navigate = useNavigate();
27    const [assets, setAssets] = useState(state?.assets ?? { deposit: '', downPayment: '', emergency: '' });
28    const [loans, setLoans] = useState(() => state?.loans ?? [createLoan()]);
29    const income = Number(state?.values?.income || 0);
30    const balance = loans.reduce((sum, loan) => sum + Number(loan.balance || 0), 0);
31    const payment = loans.reduce((sum, loan) => sum + Number(loan.payment || 0), 0);
32    const ratio = income > 0 ? payment / income * 100 : null;
33    const available = Math.max(0, Number(assets.deposit || 0) - Number(assets.emergency || 0));
34
35    if (!state?.values) return <Navigate to="/financial" replace />;
36
37    function updateLoan(id, name, value) {
38      setLoans((previous) => previous.map((loan) => loan.id === id ? { ...loan, [name]: value } : loan));
39    }
40
41    function handleSubmit(event) {
42      event.preventDefault();
43      navigate(state?.reviewReturn ? '/financial/review' : '/financial/credit', { state: { ...state, assets, loans, reviewReturn: false } });
```

각 변수의 역할

assets는 예금·선수금·비상자금이고 loans는 개별 대출 목록입니다. balance는 잔액 합계, payment는 월 상환액 합계입니다. ratio는 월 실수령액 대비 상세 상환액 비율을 나타냅니다. income이 0이면 나눗셈 대신 null을 사용해 비율을 표시하지 않습니다. 기타 정기 소득을 이 비율에 넣지 않는 것은 현재 "월 실수령액"이라는 표시 기준과 일치합니다.

available은 예금에서 비상자금을 뺀 금액입니다. 이 이름은 소득 요약의 월 가용자금과 같지만 의미와 시간 단위가 다릅니다. 구매 활용 가능 현금과 월 가용자금을 구분하는 이름을 사용하면 호출 관계를 읽을 때 혼동을 줄일 수 있습니다.

대출 수정과 전달

37-38행은 id로 대출을 찾아 해당 필드만 갱신합니다. 원본 배열과 객체를 직접 바꾸지 않아 React 상태 갱신 방식에 맞습니다. 43행에서 assets와 loans를 모두 전달하므로 대출 정보가 화면 이동 중 사라지는 것은 아닙니다. 실제 단절 지점은 전달된 loans를 다음 계산 함수가 읽지 않는 곳입니다.

진입 조건과 초기 상태

35행은 이전 소득 정보 없이 진입하면 시작 화면으로 보냅니다. 이 보호는 신용·차량·확인 화면과 통일되어 있지는 않습니다. 28행은 처음에 빈 대출 카드 하나를 생성합니다. 이 카드는 "대출이 있다"는 확인된 정보가 아니라 입력 편의를 위한 빈 폼이므로, 합계 산정과 저장 시 완성된 대출과 구분할 정책이 있으면 좋습니다.

06 차량 예산 산식과 금융 가정

실제 코드 | front/src/features/financial/calculateVehicleBudget.js:4-19

```
4  export function calculateVehicleBudget({ values = {}, assets = {}, vehicle = {} } = {}) {
5    const totals = calculateFinancialSummary(values);
6    const operatingCost = 20;
7    const annualRate = 0.06;
8    const months = vehicle.installment === '일시불' ? 0 : Number.parseInt(vehicle.installment, 10);
9    const deposit = Math.max(0, Number(assets.deposit || 0) - Number(assets.emergency || 0));
10   const downPayment = Math.min(deposit, Math.max(0, Number(assets.downPayment || 0)));
11   const monthlyPayment = Math.max(0, totals.available * 0.75 - operatingCost);
12   const monthlyRate = annualRate / 12;
13   const loan = months > 0 ? monthlyPayment * (1 - (1 + monthlyRate) ** -months) / monthlyRate : 0;
14   const budget = downPayment + loan;
15   const valid = ['income', 'housing', 'fixed', 'living'].every((key) => values[key] !== undefined && values[key] !== '')
16     && ['deposit', 'downPayment', 'emergency'].every((key) => assets[key] !== undefined && assets[key] !== '')
17     && Number.isFinite(months);
18   return { ...totals, operatingCost, annualRate, months, downPayment, monthlyPayment: months === 0 ? 0 : monthlyPayment,
19     low: downPayment + loan * 0.8, high: budget, valid };
```

선수금과 할부 원금

8행은 일시불을 0개월로, “48개월” 같은 문자열을 정수로 바꿉니다. 9행은 비상금을 제외한 현금을 계산하고, 10행은 계획한 선수금을 이 현금 범위로 제한합니다. 따라서 예금 2,000·비상금 500·선수금 계획 1,800이면 실제 계산 선수금은 1,500입니다.

11행은 월 가용자금의 25%를 남기고 월 유지비 20을 뺀 값을 할부 상환 한도로 삼습니다. 13행은 그 월 상환액을 연 6%의 월 이율과 기간으로 할인해 감당할 수 있는 대출 원금을 계산합니다. 월 상환액 × 개월 수로 단순 계산하지 않아 이자 비용이 반영됩니다.

예산 범위의 의미

high는 선수금 + 할부 가능 원금, low는 선수금 + 할부 가능 원금의 80%입니다. 이 하한은 통계적 신뢰구간이나 실제 시장 최저가가 아니라 화면 정책입니다. 일시불에서는 대출 원금이 0이므로 상·하한이 선수금으로 같아집니다. 월 할부금은 0으로 반환하지만 유지비는 계속 반영됩니다.

리뷰 판단

가정은 코드와 결과 화면에 명시되어 있으므로 6%와 20이라는 상수 자체를 오류로 분류하지 않습니다. 다만 valid는 데이터가 충분한지에 대한 플래그이며 구매가 안전하거나 대출 승인이 가능하다는 의미가 아닙니다. 결과 UI에서도 이 두 의미를 구분해야 합니다.

07 P1: 상세 상환액이 계산에서 빠지는 경로

실제 코드 | front/src/pages/FinancialAssetsPage.jsx:30-32

```
30   const balance = loans.reduce((sum, loan) => sum + Number(loan.balance || 0), 0);
31   const payment = loans.reduce((sum, loan) => sum + Number(loan.payment || 0), 0);
32   const ratio = income > 0 ? payment / income * 100 : null;
```

실제 코드 | front/src/features/financial/calculateVehicleBudget.js:4-5

```
4  export function calculateVehicleBudget({ values = {}, assets = {}, vehicle = {} } = {}) {
5    const totals = calculateFinancialSummary(values);
```

실제 코드 | front/src/features/financial/calculateFinancialSummary.js:3-7

```
3   const income = amount('income') + amount('extraIncome');
4   const expenses = ['housing', 'fixed', 'living', 'loan', 'transport', 'phone']
5     .reduce((sum, name) => sum + amount(name), 0);
6   const savings = amount('savings');
7   return { income, expenses, savings, available: income - expenses - savings };
```

코드 사이의 단절

자산 페이지에서는 loans[].payment를 합산하지만 예산 함수의 입력 분해에는 loans가 없습니다. 예산 함수가 부르는 요약 함수도 values.loan만 읽습니다. 즉 대출을 입력하고 전달하는 코드는 있는데 최종 계산이 그 데이터를 소비하지 않습니다. 자산 화면의 월 부채부담 요약이 정상으로 보이기 때문에 사용자가 누락을 알아차리기 어렵습니다.

동일한 기본 조건	상세 월 상환 0	상세 월 상환 150
소득 / 대출 외 지출	300 / 130	300 / 130
현재 계산: 가용자금	170	170
현재 계산: 예산 상한	약 5,577.4	약 5,577.4
상세 상환을 반영할 때: 가용자금	170	20
같은 정책으로 재계산한 상한	약 5,577.4	1,000

선수금 1,000·예금 2,000·비상금 500·48개월 조건입니다. 현재 결과 두 건은 이전 Node.js 실행에서 확인했습니다. 상세 상환 반영 시의 값은 동일 산식으로 설명한 기대 결과이며, 실제 금융 한도는 아닙니다.

수정 시 주의점

values.loan과 상세 합계를 단순히 더하면 같은 대출이 두 번 차감될 수 있습니다. 상세 내역을 기준으로 삼을지, 총액 입력을 기준으로 삼을지 하나를 정해야 합니다. 예산뿐 아니라 확인 화면의 가용자금, 결과의 대출 분포와 구매 전후 월 상환액까지 함께 같은 기준으로 변경해야 합니다.

08 P2: 자산 화면과 결과 검증의 불일치

실제 코드 | front/src/pages/FinancialAssetsPage.jsx:52-61

```
52            <input
53              id={id}
54              name={id}
55              type="number"
56              min="0"
57              step={step}
58              inputMode={name === 'months' ? 'numeric' : 'decimal'}
59              placeholder="0"
60              value={loan ? loan[name] : assets[name]}
61              aria-label={`${label} (${unit})`}
```

실제 코드 | front/src/pages/FinancialAssetsPage.jsx:41-44

```
41    function handleSubmit(event) {
42      event.preventDefault();
43      navigate(state?.reviewReturn ? '/financial/review' : '/financial/credit', { state: { ...state, assets, loans, reviewReturn: false } });
44    }
```

실제 코드 | front/src/features/financial/calculateVehicleBudget.js:15-17

```
15    const valid = ['income', 'housing', 'fixed', 'living'].every((key) => values[key] !== undefined && values[key] !== '')
16      && ['deposit', 'downPayment', 'emergency'].every((key) => assets[key] !== undefined && assets[key] !== '')
17      && Number.isFinite(months);
```

왜 입력을 마쳤는데 결과가 비어 있는가

입력칸에는 min과 step은 있지만 required가 없습니다. 자산 제출 함수도 값이 비었는지 확인하지 않고 다음 단계로 이동합니다. 반면 결과 계산은 예금·선수금·비상금 모두 빈 문자열이 아니어야 valid=true로 판정합니다. 비상금이 없는 사용자가 빈칸으로 두면 정상 이동 후 결과에서만 입력 부족으로 처리됩니다.

더구나 금액 계산의 Number(assets.emergency || 0)은 빈 값을 0으로 계산합니다. 계산에는 0으로 사용하면서 표시 가능 여부에서는 부족한 정보로 판단하는 이중 규칙입니다. 이전 함수 재현에서 emergency=""일 때 숫자 예산이 계산되어도 valid=false인 것을 확인했습니다.

권장 정책과 구현 지점

0이 가능하지만 반드시 입력해야 하는 항목이면 자산 화면에 required와 필수 안내를 추가합니다. 선택 입력이라면 제출 경계에서 0으로 정규화하고 결과의 필수값 검사도 조정합니다. 빈칸이 없는지와 유효한 숫자인지는 다른 조건이므로, 비숫자·유한성·음수 검사도 같은 정책 안에서 다루는 편이 명확합니다.

완료 기준

입력 화면이 정상 제출을 허용한 데이터는 결과에서도 유효해야 합니다. 빈 값과 명시적 0의 처리, 수정 후 복귀, 세 자산 항목을 각각 비운 경우를 검증합니다. 선수금이 사용 가능 현금을 넘는 경우에는 현재의 상한 제한을 유지하되 사용자에게 조정된 금액을 이해할 수 있게 안내합니다.

09 신용정보 입력과 기본값의 의미

실제 코드 | front/src/pages/FinancialCreditPage.jsx:7-15

```
7  const overdueOptions = ['연체 없음', '30일 미만', '30일 이상', '잘 모르겠어요'];
8  const countOptions = ['0', '1', '2', '3', '4', '5 이상', '잘 모르겠어요'];
9  const creditFields = [
10   { name: 'overdueCount', label: '최근 1년 연체 횟수', options: countOptions, unit: '회' },
11   { name: 'newLoans', label: '최근 6개월 신규 대출', options: countOptions, unit: '건' },
12   { name: 'cards', label: '보유 카드 수', options: countOptions, unit: '개' },
13 ];
14 const initialCredit = { overdue: '연체 없음', overdueCount: '0', newLoans: '1', cards: '2', firstCredit: '2021-03', nice: '', kcb: '' };
15 const scoreOptions = ['900~1,000점', '800~899점', '700~799점', '600~699점', '600점 미만'];
```

실제 코드 | front/src/pages/FinancialCreditPage.jsx:29-39

```
29 export default function FinancialCreditPage() {
30   const { state } = useLocation();
31   const navigate = useNavigate();
32   const [credit, setCredit] = useState(() => ({ ...initialCredit, ...state?.credit }));
33   const completed = ['overdue', 'overdueCount', 'newLoans', 'cards', 'firstCredit'].filter((name) => credit[name] !== '').length;
34   const [currentMonth] = useState(() => {
35     const today = new Date();
36     return `${today.getFullYear()}-${String(today.getMonth() + 1).padStart(2, '0')}`;
37   });
38   const updateCredit = (name, value) => {
39     setCredit((previous) => ({ ...previous, [name]: value }));
```

입력 모델의 특징

연체·신규 대출·카드 수는 문자열 선택값이고, 일부는 숫자가 아닌 "5 이상"과 "잘 모르겠어요"입니다. 따라서 뒤에서 단순 Number 변환만으로 처리하면 모든 상태를 표현할 수 없습니다. 화면의 문구를 데이터 코드로 겸용하므로 표시 문구가 변경될 때 계산 조건도 영향을 받습니다.

기본값과 완료율

initialCredit에는 연체 없음·연체 0회·신규 대출 1건·카드 2개·최초 거래 2021-03이 이미 들어 있습니다. 32행은 저장된 값이 없으면 이 값을 사용하고, 33행은 빈 문자열 여부만 세므로 최초 진입부터 필수 5개가 완료로 집계됩니다. 코드상 "선택값이 있음"을 세며 사용자가 직접 확인했다는 뜻은 아닙니다.

이 부분은 예시 데이터로 화면을 구성하는 단계에서는 가능하지만, 개인 분석으로 전환하면 다른 사람의 이력처럼 보이는 기본값이 실제 입력으로 사용될 수 있습니다. 이번의 확정 결함 3건과 별개로, 미입력 상태와 사용자가 확인한 값을 구분하는 제품 정책을 권장합니다.

개선 방향

선택지의 표시 문구와 저장 코드를 분리하거나 작은 해석 함수를 둡니다. 최초 거래 시점은 임의 날짜 대신 미입력으로 시작하고, 거래 이력이 없는 상태도 표현할 필요가 있는지 확인합니다. 완료율은 유효한 응답을 셀지, 사용자가 확인한 항목을 셀지 정의해야 합니다.

10 P2: 신용 미확인을 0회처럼 처리

실제 코드 | front/src/pages/FinancialResultPage.jsx:23-31

```
23    const higher = [];
24    const lower = [];
25    if (credit.overdue && credit.overdue !== '연체 없음') higher.push(credit.overdue === '잘 모르겠어요' ? '현재 연체 여부 확인이 필요합니다.' : `현재 ${credit.overdue} 연체가 있습니다.`);
26    if (credit.overdue === '연체 없음') lower.push('현재 연체 중인 금액이 없습니다.');
27    if (Number(credit.newLoans) > 0 || credit.newLoans === '5 이상') higher.push('최근 6개월 이내 신규 대출이 있습니다.');
28    if (Number(credit.overdueCount) > 0 || credit.overdueCount === '5 이상') higher.push('최근 1년 이내 연체 이력이 있습니다.');
29    if (credit.firstCredit) lower.push(`최초 신용거래 시점은 ${credit.firstCredit.replace('-', '년 ')}월입니다.`);
30    if (result.available <= 0 && state?.values) higher.push('월 가용자금이 없어 지출·상환 계획 확인이 필요합니다.');
31    const status = !state?.credit ? '입력 필요' : higher.length ? '확인 필요' : '특이사항 없음';
```

조건문을 실제 값으로 따라가기

newLoans와 overdueCount가 "잘 모르겠어요"이면 Number 변환 결과는 NaN입니다. NaN > 0은 false이고 "5 이상"과도 같지 않아 higher 배열에 문구가 추가되지 않습니다. 현재 연체는 "연체 없음", 가용자금은 양수인 조건이면 higher가 비어 최종 status는 "특이사항 없음"이 됩니다.

25행은 현재 연체 여부의 미확인을 명시적으로 처리합니다. 같은 "잘 모르겠어요" 응답이 항목마다 서로 다른 의미로 처리되는 셈입니다. 정보를 모르는 상태는 이력이 확인되지 않은 상태이지, 이력이 0으로 확인된 상태는 아닙니다.

개선 예시 | 설명용 제안 코드, 프로젝트 미적용

```
const unknownFields = [
  ['newLoans', '최근 6개월 신규 대출'],
  ['overdueCount', '최근 1년 연체 횟수'],
].filter(([key]) => credit[key] === '잘 모르겠어요');
for (const [, label] of unknownFields) {
  higher.push(`${label} 확인이 필요합니다.`);
}
```

예시를 적용할 때의 판단

위 코드는 기존 higher 안내에 미확인 항목을 추가하는 최소 방향입니다. 실제 설계에서는 확인된 이력과 정보 부족 목록을 분리하면 "대출이 있음"과 "대출 여부를 모름"을 더 정확히 표현할 수 있습니다. 미확인을 신용 불량이나 금융기관의 거절 판정으로 해석해서는 안 됩니다.

추가 관찰

29행은 최초 거래 날짜가 존재한다는 이유만으로 긍정 요인에 넣습니다. 경과 기간을 평가하는 로직은 아닙니다. 또한 카드 수와 NICE/KCB 값은 이 판정 코드에서 사용되지 않습니다. 입력한 모든 정보가 종합 점수에 반영되는 것처럼 설명하지 않도록 현재 기능 범위와 문구를 맞추는 편이 좋습니다.

11 차량 조건: 저장·표시와 계산의 경계

실제 코드 | front/src/pages/FinancialVehiclePage.jsx:30-42

```
30  export default function FinancialVehiclePage() {
31    const { state } = useLocation();
32    const navigate = useNavigate();
33    const [vehicle, setVehicle] = useState(() => ({ ...initialVehicle, ...state?.vehicle }));
34    const [priorities, setPriorities] = useState(() => state?.priorities ?? ['월 총비용', '연비', '차량가격', '안전·편의', '실내공간']);
35    const [notice, setNotice] = useState('');
36    const profile = profiles[vehicle.purpose];
37    const recommendedFuel = vehicle.fuel === '가솔린 · 하이브리드' ? '하이브리드' : vehicle.fuel === '상관없음' ? '선호 없음' : vehicl
    e.fuel;
38
39    function handleSubmit(event) {
40      event.preventDefault();
41      navigate('/financial/review', { state: { ...state, vehicle, priorities, reviewReturn: false } });
42    }
```

실제 코드 | front/src/pages/FinancialVehiclePage.jsx:83-88

```
83                <dl>
84                  <div><dt>연간 예상 주행</dt><dd>{vehicle.annualDistance}</dd></div>
85                  <div><dt>월 환산 주행</dt><dd>{Math.round(Number(vehicle.annualDistance.replace(/[^0-9]/g, '')) / 12).toLocaleStri
    ng('ko-KR')}km</dd></div>
86                  <div><dt>선호 할부</dt><dd>{vehicle.installment}</dd></div>
87                  <div><dt>추천 연료</dt><dd>{recommendedFuel}</dd></div>
88                </dl>
```

실제 코드 | front/src/features/financial/calculateVehicleBudget.js:6-8

```
6   const operatingCost = 20;
7   const annualRate = 0.06;
8   const months = vehicle.installment === '일시불' ? 0 : Number.parseInt(vehicle.installment, 10);
```

실제로 반영되는 항목

vehicle과 priorities는 state로 유지되어 확인 화면으로 전달됩니다. purpose는 profiles에서 설명 문구를 고르는 데 사용되고, annualDistance는 월 환산 주행거리 표시에 쓰입니다. fuel은 추천 연료 문구를 선택합니다. 예산 함수에서 사용하는 차량 조건은 installment이며, 유지비는 연료·주행거리와 무관하게 20으로 고정됩니다.

현재 구현을 어떻게 해석해야 하는가

입력한 차량 조건이 사라진 것은 아닙니다. 저장과 화면 표시는 구현되어 있지만, 실제 차량 검색이나 차량별 유지비 계산으로 이어지는 경로는 이 코드에 없습니다. 따라서 선택 UI 구현 완료와 추천 엔진 반영 완료를 구분해야 합니다. 같은 재무 조건에서 연료나 주행거리를 바꿔도 현재 예산은 동일합니다.

개선 방향

현재 단계에서는 "조건에 따라 월 유지비를 계산한다"는 안내를 실제 구현 범위에 맞춥니다. 차량 가격·연비·보험·세금 등 계산에 필요한 데이터와 산식이 준비되면 별도의 유지비 함수로 분리하고 예산 함수에 결과를 전달하는 방식이 적절합니다. 데이터가 없는 상태에서 임의의 정밀한 비용표를 추가할 필요는 없습니다.

화면 문구 "48개월", "15,000km"를 계산 시 해석하는 대신 저장값은 48, 15000 같은 수치로 두고 표시에만 단위를 붙이는 방안도 있습니다. 이 변경은 현재 즉시 장애를 고치는 항목이라기보다 향후 API·모델 입력 계약을 명확히 하는 개선입니다.

12 우선순위 정렬 함수와 키보드 동작

실제 코드 | front/src/features/financial/reorderPriorities.js:1-9

```
1  export function reorderPriorities(items, source, target) {
2    const from = items.indexOf(source);
3    const to = items.indexOf(target);
4    if (from < 0 || to < 0 || from === to) return items;
5    const reordered = [...items];
6    reordered.splice(from, 1);
7    reordered.splice(to, 0, source);
8    return reordered;
9  }
```

실제 코드 | front/src/features/financial/VehiclePriorities.jsx:9-14

```
 9    function move(source, target) {
10      const reordered = reorderPriorities(priorities, source, target);
11      if (reordered === priorities) return;
12      onChange(reordered);
13      setAnnouncement(`${source}, ${reordered.indexOf(source) + 1}순위로 이동했습니다.`);
14    }
```

실제 코드 | front/src/features/financial/VehiclePriorities.jsx:88-95

```
88                  if (event.key === 'Escape') { cancelDrag(); return; }
89                  const direction = ['ArrowLeft', 'ArrowUp'].includes(event.key) ? -1 : ['ArrowRight', 'ArrowDown'].includes(event.key) ? 1 :
    0;
90                  if (!direction) return;
91                  event.preventDefault();
92                  if (priorities[index + direction]) move(item, priorities[index + direction]);
93                }}
94              >
95                <span className="vehicle-drag-handle" aria-hidden="true">  </span>
```

배열 변경과 화면 갱신

정렬 함수는 source와 target의 인덱스를 찾고, 없는 항목이나 동일 위치이면 원래 배열을 반환합니다. 이동이 필요할 때만 복사본을 만들어 source를 제거하고 target의 기존 인덱스에 삽입합니다. 원본을 바꾸지 않아 props로 받은 배열을 안전하게 다룹니다.

예를 들어 [A, B, C]에서 A를 B로 이동하면 [B, A, C]가 됩니다. move는 같은 배열 참조가 돌아오면 변경이 없는 것으로 보고 종료합니다. 키보드 방향키는 현재 항목의 이웃을 목표로 넘기므로 한 칸 이동하는 규칙과 맞습니다.

접근성 측면에서의 장점

방향키 기본 동작을 막고 목록 순서를 바꿀 수 있으며 이동 결과를 announcement로 알립니다. 마우스 드래그만 제공하는 구현보다 조작 수단이 다양합니다. 실제 포커스 유지와 스크린리더 안내는 브라우저 검증이 필요하므로 코드 존재만으로 접근성이 완성되었다고 판단하지는 않습니다.

설계 개선 제안

키보드는 reorderPriorities를 호출하지만 포인터 드롭은 별도 삽입 코드를 사용합니다. 두 경로의 의도는 비슷해도 포인터에는 앞·뒤 삽입이라는 추가 의미가 있으므로 무조건 같은 함수로 합칠 필요는 없습니다. 공통화한다면 "이동 후 최종 삽입 인덱스"를 명시하는 작은 함수가 경계를 더 분명하게 만듭니다.

13 드래그의 시작·목표 탐색·종료

실제 코드 | front/src/features/financial/VehiclePriorities.jsx:16-26

```
16  function startDrag(event, item) {
17    if (!event.isPrimary || event.button !== 0) return;
18    const rect = event.currentTarget.getBoundingClientRect();
19    gesture.current = { source: item, target: null, x: event.clientX, y: event.clientY, offsetX: event.clientX - rect.left, offsetY: event.clien
    tY - rect.top, width: rect.width, height: rect.height, moved: false };
20    event.currentTarget.setPointerCapture(event.pointerId);
21  }
22
23  function updateDrag(event) {
24    const current = gesture.current;
25    if (!current) return;
26    if (!current.moved && Math.hypot(event.clientX - current.x, event.clientY - current.y) < 5) return;
```

실제 코드 | front/src/features/financial/VehiclePriorities.jsx:40-47

```
40      const nearest = candidates[0];
41      if (nearest) {
42        current.target = nearest.element.dataset.vehiclePriority;
43        current.after = event.clientX > nearest.rect.left + nearest.rect.width / 2;
44      }
45    }
46    setDrag({ source: current.source, target: current.target, after: current.after,
47      left: event.clientX - current.offsetX, top: event.clientY - current.offsetY, width: current.width, height: current.height });
```

실제 코드 | front/src/features/financial/VehiclePriorities.jsx:50-61

```
50  function finishDrag(event) {
51    const current = gesture.current;
52    if (current?.moved && current.target) {
53      const reordered = priorities.filter((item) => item !== current.source);
54      const insertion = reordered.indexOf(current.target) + (current.after ? 1 : 0);
55      reordered.splice(insertion, 0, current.source);
56      onChange(reordered);
57      setAnnouncement(`${current.source}, ${insertion + 1}순위로 이동했습니다.`);
58    }
59    gesture.current = null;
60    setDrag(null);
61    if (event.currentTarget.hasPointerCapture(event.pointerId)) event.currentTarget.releasePointerCapture(event.pointerId);
```

포인터 처리의 의미

시작 시 주 포인터와 왼쪽 버튼만 받아들이고 pointer capture를 설정합니다. 원래 위치와 클릭 오프셋을 저장하므로 항목을 누른 지점을 기준으로 미리보기를 이동시킬 수 있습니다. 이동 거리가 5px 미만이면 드래그를 시작하지 않아 단순 클릭의 흔들림을 줄입니다.

updateDrag의 중간 부분은 목록 안의 자기 자신을 제외한 항목과 포인터 사이 거리를 구해 가장 가까운 목표를 고릅니다. 발췌한 43행은 목표의 가로 중앙을 기준으로 앞·뒤 삽입을 결정합니다. 종료 시 source를 먼저 뺀 배열에서 삽입 위치를 구하므로 제거에 따른 인덱스 이동을 고려합니다.

검증할 경계

목록이 여러 줄로 감싸질 때 행 시작·끝의 삽입 의도가 자연스러운지, 목록 밖 드롭 시 원래 순서를 유지하는지 확인해야 합니다. 취소·캡처 해제 시 gesture와 drag를 지우는 경로는 존재합니다. 다중 포인터, 스크롤과 터치 조작, Escape 취소는 실제 브라우저에서 확인할 항목이며 현재 결함으로 단정하지 않습니다.

14 확인 화면과 결과 표현의 책임

실제 코드 | front/src/pages/FinancialReviewPage.jsx:24-36

```
24  const { values = {}, assets = {}, loans, credit = {}, vehicle = {}, priorities = [] } = state ?? {};
25  const totals = calculateFinancialSummary(values);
26  const edit = (path) => navigate(path, { state: { ...state, reviewReturn: true } });
27  const incomeRows = [
28    ['월 실수령액', money(values.income)], ['기타 정기 소득', money(values.extraIncome)],
29    ['월 주거비', money(values.housing)], ['월 고정 지출', money(values.fixed)], ['월 생활비', money(values.living)],
30    ['현재 대출 월 상환액', money(values.loan)], ['교통비', money(values.transport)], ['통신비', money(values.phone)],
31    ['월 지출·상환 합계', state?.values ? money(totals.expenses) : '입력 필요'], ['목표 저축', money(values.savings)],
32    ['월 가용자금', state?.values ? money(totals.available) : '입력 필요'],
33  ];
34  const assetRows = [
35    ['사용 가능 예금', money(assets.deposit)], ['선수금', money(assets.downPayment)], ['비상자금', money(assets.emergency)],
36    ['월 부채상환', loans ? money(loans.reduce((sum, loan) => sum + Number(loan.payment || 0), 0)) : '입력 필요'],
```

실제 코드 | front/src/pages/FinancialResultPage.jsx:32-37

```
32  const purchaseAvailable = result.available - result.monthlyPayment - result.operatingCost;
33  const beforeAfter = [
34    { label: '가용자금', before: result.available, after: purchaseAvailable },
35    { label: '월 상환액', before: Number(values.loan || 0), after: Number(values.loan || 0) + result.monthlyPayment },
36    { label: '목표 저축', before: result.savings, after: result.savings },
37  ];
```

확인 화면이 드러내는 데이터 불일치

확인 화면은 values로 재무 요약을 계산하면서 자산 카드에는 loans 합계를 따로 보여줍니다. 따라서 "현재 대출 월 상환액"과 "월 부채상환"이 서로 다르게 표시될 수 있습니다. 두 카드의 표현 문제가 아니라 7쪽에서 분석한 상환액 기준 분리가 화면까지 전파된 사례입니다.

구매 전후 비교의 의미

purchaseAvailable은 구매 전 가용자금에서 새 할부금과 유지비를 빼고, beforeAfter는 같은 값을 표와 그래프에 공급합니다. 한 비교 배열을 재사용하는 것은 일관성에 도움이 됩니다. 기존 월 상환액은 여전히 values.loan에서 읽으므로 예산만 고치고 여기를 그대로 두면 비교 표는 계속 잘못될 수 있습니다.

컴포넌트 책임과 읽기 쉬운 구조

FinancialResultPage는 계산 호출, 신용 안내 판단, 그래프 데이터 구성, 화면 표시를 함께 수행합니다. 계산이 커지면 React에 의존하지 않는 신용정보 해석 함수부터 분리하는 편이 좋습니다. 단순 마크업까지 무조건 분리하기보다 변경 이유가 다른 정책을 우선 분리한다는 프로젝트 설계 기준에 맞습니다.

ReviewCard처럼 제목·수정 버튼·정보 목록이라는 독립적인 UI 의미가 있는 요소의 분리는 적절합니다. FinancialProgress는 다른 단계에서 재사용되므로 페이지 파일에서 export하기보다 financial 기능 컴포넌트로 두는 것도 고려할 수 있습니다. 이는 현재 동작 오류가 아닌 배치 개선입니다.

15 상환액 통합의 제안 코드

아래 예시는 상세 대출을 상환액의 기준으로 정한다는 정책을 전제로 합니다. 기존 총액 입력과 상세 대출을 더하지 않습니다. 실제 적용 시 첫 단계의 대출 총액 입력도 제거하거나 동일 합계로 연동해야 합니다.

개선 예시 | 설명용 제안 코드, 프로젝트 미적용

```
export function normalizeFinancialInput(state = {}) {
  const { values = {}, loans = [] } = state;
  const monthlyLoanPayment = loans.reduce(
    (sum, loan) => sum + Number(loan.payment || 0), 0
  );
  return {
    ...values,
    loan: monthlyLoanPayment,
  };
}

// 예산 / 확인 / 결과에서 같은 값을 사용한다.
const financialValues = normalizeFinancialInput(state);
const totals = calculateFinancialSummary(financialValues);
const result = calculateVehicleBudget({
  ...state,
  values: financialValues,
});
```

왜 호출 지점을 함께 바꿔야 하는가

예산 함수 안에서만 loans를 합산하면 결과 그래프가 values.loan을 읽는 문제와 확인 화면의 가용자금 문제는 남습니다. 위 예시의 목적은 정규화된 재무 입력 하나를 모든 소비자에게 전달하는 것입니다. 결과 페이지의 distribution과 beforeAfter도 financialValues.loan을 사용하도록 맞춰야 합니다.

실제 적용 전에 보완할 계약

입력 경계에서 유한한 0 이상 금액인지 검증한 뒤 합산해야 합니다. 위 코드는 정상 검증된 대출 입력을 전제로 하며 비숫자 값의 오류 처리까지 구현한 완성 패치가 아닙니다. 저장 데이터 호환이 필요하다면 "상세 대출을 아직 입력하지 않음"과 "대출 없음으로 확인함"을 구분해 기존 values.loan을 언제 대체할지도 정해야 합니다.

자산 검증과 같은 원칙

자산도 UI 제출과 예산 함수가 서로 다른 규칙을 갖지 않도록 한 곳에서 계약을 정합니다. 오류가 있으면 계산 결과를 내기 전에 해당 필드의 오류를 반환하고, 정상 입력일 때 숫자로 정규화하면 빈 값 검사와 실제 계산의 의미가 일치합니다.

16 검증 근거와 수정 우선순위

항목	현재 확인 수준	후속 검증
P1 상세 상환액 누락	코드 추적 + 이전 Node 계산 재현	대출 추가·수정·삭제와 중복 차감 방지
P2 자산 검증 불일치	코드 추적 + 이전 valid=false 재현	빈 값·0·음수와 화면 이동
P2 신용 미확인 누락	정적 조건 분기 확인	0 / 1 / 5 이상 / 미확인 선택 조합
우선순위 조작	구현 경로 검토	키보드·드래그·취소·여러 줄 배치
전체 빌드	이전 실행 실패	lucide-react 의존성 해석 복구 후 재실행

이전 계산 재현의 핵심 입력

개선 예시 | 설명용 제안 코드, 프로젝트 미적용

```
const base = {
  values: { income: '300', housing: '50', fixed: '30', living: '50' },
  assets: { deposit: '2000', downPayment: '1000', emergency: '500' },
  vehicle: { installment: '48개월' },
};
calculateVehicleBudget({ ...base, loans: [{ payment: '0' }] });
calculateVehicleBudget({ ...base, loans: [{ payment: '150' }] });
// 현재 두 호출 모두 available=170, high 5577.3842, valid=true
```

위 코드는 수정 예시가 아니라 이전 재현 조건을 문서에 정리한 실행 예입니다. 별도 자동 테스트 파일을 추가하거나 이번 문서 작성에서 재실행한 것은 아닙니다.

권장 수정 순서

먼저 상환액 기준을 통합하고 요약·예산·확인·결과 전체를 함께 수정합니다. 다음으로 자산의 필수·선택 정책을 일치시키고, 신용의 미확인 상태를 보완합니다. 이후 실제 입력 확인을 요구하는 기본값 정책, 차량 조건의 계산 반영 범위, 상태 진입 경계를 정리하면 됩니다.

검증의 제한

이전 Vite 빌드는 LoginPage.jsx에서 lucide-react import를 해석하지 못해 실패했습니다. 따라서 전체 빌드 통과를 주장할 수 없습니다. 브라우저 E2E, 모바일·접근성, API·ML 연동은 미검증입니다. 고정 금융 가정 자체와 준비 중으로 명시된 추천 기능은 이번 확정 결함 3건과 구분했습니다.

검토 기준: docs/design.md, docs/naming.md 및 앞선 리뷰에서 확인한 UI·테스트 기준. 코드 출처는 각 발췌 위의 저장소 경로와 줄 번호로 표시했습니다. 문서의 코드 줄바꿈은 지면에 맞춘 표시이며 원본 소스를 변경하지 않았습니다.